What Helps a Small CNN Generalize? Batch Normalization and Data Augmentation on Dogs vs. Cats

Lianjie (Marc) Shi · Deep Learning and Artificial Intelligence, Peking University · October 2026

Code: github.com/Marclianjie/data-science-portfolio/tree/main/cnn-ablation-dogs-vs-cats

This report is a code-free version of the project notebook. Every number, table and figure below is an output of that notebook; figures with untranslated labels were redrawn from the logged values.

Overview

The task is binary image classification (cat vs. dog) with a complete PyTorch pipeline: data loading, model construction, training and inference. Within that pipeline the project compares four ways of building the same convolutional network: (1) neither batch normalization (BN) nor data augmentation (DA), (2) BN only, (3) DA only and (4) both. Initialization, normalization and augmentation address different problems: a good initialization keeps signals from vanishing or exploding at the start of training, normalization keeps the scale of intermediate activations under control during training, and augmentation creates new label-preserving versions of the training images, which effectively enlarges the training set.

Reproducibility. All random seeds (Python, NumPy, PyTorch and CUDA) are fixed to 1 and cuDNN runs in deterministic mode, so the initial weights and the order of training batches are identical across runs. Training used PyTorch 1.8.1 on a CUDA GPU.

1. Data

1.1 Dataset and folder structure

The images come from the 2013 Kaggle *dogs-vs-cats* competition. The data are already organized as `train/` and `validation/`, each split into `cats/` and `dogs/`, so the folder names serve as labels and `torchvision.datasets.ImageFolder` builds the labeled dataset directly (cats → 0, dogs → 1).

Split	Cats	Dogs	Total
Training	7,500	7,500	15,000
Validation	5,000	5,000	10,000

Table 1. Image counts by split and class (25,000 images; classes balanced).

1.2 Transforms with and without augmentation

- **Without DA** (baseline): resize to 128 × 128, then convert to a tensor in [0, 1].
- **With DA**: after resizing, a random 128 × 128 crop with 8-pixel padding (a small random shift) and a random horizontal flip. Neither changes what the image shows, so each pass over the data sees new, label-preserving versions of the training images.
- **Validation images are never augmented**; they are only resized and used for evaluation.

1.3 Datasets and data loaders

Training and validation loaders use a batch size of 128. The training loader shuffles with its own seeded generator, so the batch order is reproducible; the validation loader does not shuffle. One training batch has shape

[128, 3, 128, 128] with 128 labels.

1.4 Data display



Figure 1. Sixteen images from one augmented training batch.

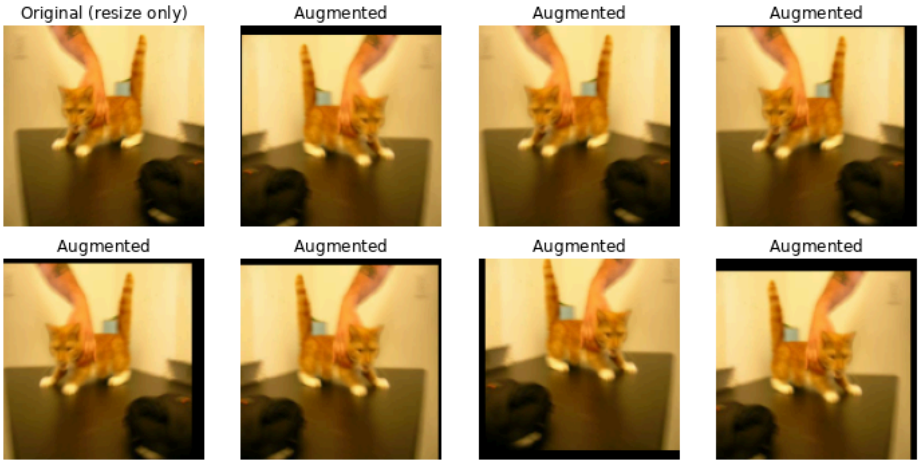


Figure 2. One training image (top left, resized only) and seven random augmentations of it.

Analysis. The dataset has 25,000 images: 15,000 for training and 10,000 for validation, with cats and dogs at 50% each, so the classes are balanced. Because the data are stored as class folders, `ImageFolder` yields (image, label) pairs without a hand-written Dataset class. Figure 2 shows how random cropping and flipping turn one cat photo into several different images with the same label, which is why augmentation acts like a larger training set and improves generalization.

2. Model

2.1 A custom CNN with optional BN and Kaiming initialization

The network has three convolutional blocks (convolution → BN → ReLU) and two max-pooling layers, followed by a fully connected head that outputs two logits. When `use_bn=True`, a `BatchNorm2d` layer follows every convolution. Weights use Kaiming initialization (normal, fan-out mode for convolutions; uniform for linear layers; zero biases), which keeps the variance of each layer's output stable and so avoids vanishing signals and gradients early in training.

Layer	Output shape	Parameters
Conv 5 × 5, stride 2 (3 → 16)	16 × 62 × 62	1,216
BatchNorm	16 × 62 × 62	32
Conv 3 × 3, stride 2 (16 → 32)	32 × 30 × 30	4,640
BatchNorm	32 × 30 × 30	64
Max-pool 2 × 2	32 × 15 × 15	0
Conv 3 × 3, stride 1 (32 → 64)	64 × 13 × 13	18,496
BatchNorm	64 × 13 × 13	128
Max-pool 2 × 2	64 × 6 × 6	0
Linear (2304 → 128), ReLU, dropout 0.5	128	295,040
Linear (128 → 2)	2	258
Total		**319,874**

Table 2. Model summary for the BN version (from `torchsummary`). Every parameter is trainable.

2.2 Four configurations and parameter counts

Configuration	BatchNorm	Augmentation	Parameters
(1) Baseline	×	×	319,650
(2) BN only	✓	×	319,874
(3) DA only	×	✓	319,650
(4) BN + DA	✓	✓	319,874

Table 3. The 2 × 2 design. BN is switched on and off in the model; DA is switched on and off in the training transform. The two switches are independent, so one model class covers all four configurations.

Parameter check. The counts were computed by hand layer by layer and match the summary: conv1 = 3 × 16 × 5 × 5 + 16 = 1,216; conv2 = 4,640; conv3 = 18,496; the three BN layers 32 + 64 + 128 = 224; fc1 = 2304 × 128 + 128 = 295,040; fc2 = 258; total 319,874.

- With about 320,000 parameters, this is a very light CNN.
- Each layer's count is set by its input and output sizes; for example, conv1 has 3 input channels × 16 output channels × a 5 × 5 kernel plus 16 biases = 1,216.
- **BN adds very few parameters** (a learnable scale and shift per channel, 224 in total, or 0.07% of the model), yet it changes how hard the network is to optimize.
- **The fully connected layers hold most of the parameters** (295,298, or 92.3%), typical of a small feature map flattened into a wide vector; they would be the first place to cut if the model had to be smaller.
- Without BN the model has 319,874 − 224 = 319,650 parameters, matching the code's output.

3. Training the four configurations

- **Loss:** cross-entropy (softmax included). **Optimizer:** Adam with learning rate 10^{-3}. **Epochs:** 30.
- **Monitoring:** training loss and validation accuracy, recorded after every epoch.

- All four models start from the **same random initialization and see the same batch order**, so any difference comes from BN and DA alone.

Epoch	(1) Baseline		(2) BN only		(3) DA only		(4) BN + DA	
	Loss	Val. acc.	Loss	Val. acc.	Loss	Val. acc.	Loss	Val. acc.
1	0.6663	62.54%	0.6932	62.40%	0.6739	56.53%	0.7103	66.03%
2	0.5893	71.41%	0.5978	70.05%	0.6129	72.80%	0.6036	60.04%
3	0.5310	76.12%	0.5546	72.11%	0.5355	77.30%	0.5758	70.01%
4	0.4864	76.02%	0.5180	75.71%	0.4932	75.16%	0.5532	70.27%
5	0.4466	78.67%	0.4882	76.05%	0.4715	80.30%	0.5260	71.72%
6	0.4199	78.31%	0.4614	77.91%	0.4466	78.63%	0.5059	72.93%
7	0.3856	79.48%	0.4299	78.88%	0.4328	80.06%	0.4762	75.89%
8	0.3635	80.99%	0.4166	66.32%	0.4138	81.06%	0.4701	65.51%
9	0.3250	81.59%	0.4041	79.42%	0.4037	81.96%	0.4559	78.92%
10	0.2983	82.00%	0.3863	78.56%	0.4069	82.67%	0.4408	69.54%
11	0.2705	82.21%	0.3638	72.32%	0.3820	83.11%	0.4310	78.09%
12	0.2673	81.61%	0.3592	74.83%	0.3834	83.05%	0.4203	80.91%
13	0.2181	80.97%	0.3288	80.95%	0.3641	84.30%	0.4103	81.79%
14	0.2014	81.89%	0.3115	81.94%	0.3601	83.02%	0.4096	70.02%
15	0.1737	81.78%	0.2947	79.69%	0.3541	82.28%	0.3980	80.35%
16	0.1618	80.80%	0.2727	79.22%	0.3471	81.58%	0.3985	82.79%
17	0.1411	81.66%	0.2731	80.83%	0.3411	84.80%	0.3818	81.86%
18	0.1253	80.53%	0.2498	81.47%	0.3372	85.12%	0.3819	81.24%
19	0.1085	81.67%	0.2367	82.82%	0.3293	85.37%	0.3792	82.72%
20	0.0933	82.05%	0.2147	73.86%	0.3269	86.10%	0.3664	84.44%
21	0.0809	81.39%	0.2108	82.85%	0.3184	85.52%	0.3646	84.80%
22	0.0757	81.60%	0.1894	82.51%	0.3053	86.47%	0.3568	78.52%
23	0.0694	81.57%	0.1856	77.47%	0.3016	86.64%	0.3560	86.02%
24	0.0563	81.98%	0.1719	78.69%	0.3060	86.81%	0.3530	85.51%
25	0.0643	81.83%	0.1499	78.20%	0.2925	86.76%	0.3410	85.43%
26	0.0487	81.40%	0.1579	82.91%	0.2888	85.39%	0.3427	86.36%
27	0.0515	82.11%	0.1618	82.82%	0.2813	86.69%	0.3382	85.99%
28	0.0432	81.02%	0.1368	80.88%	0.2718	87.54%	0.3320	86.23%
29	0.0459	80.89%	0.1260	78.92%	0.2771	84.70%	0.3238	83.91%
30	0.0481	80.34%	0.1236	82.51%	0.2812	87.64%	0.3213	82.47%

Table 4. Training log: training loss and validation accuracy after each epoch.

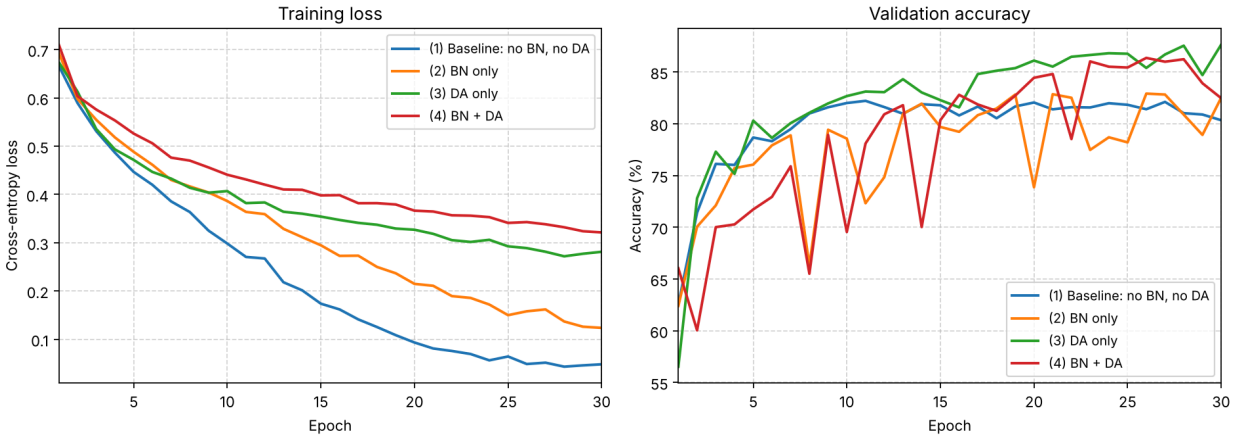


Figure 3. Training loss (left) and validation accuracy (right) by epoch.

Analysis. After 30 epochs the final validation accuracies are (1) 80.34%, (2) 82.51%, (3) 87.64% and (4) 82.47%.

1. **Effect of BN (1 vs. 2).** BN alone is 2.17 points above the baseline (82.51% vs. 80.34%), and the gain comes mainly from **stability late in training**. The baseline peaks at 82.21% in epoch 11 and then keeps falling, ending 1.87 points below its peak, while the BN model peaks at 82.91% in epoch 26 and ends only 0.40 points below it. Normalizing intermediate features damps late-training oscillation, but early convergence is similar (78.67% vs. 76.05% at epoch 5).
2. **Effect of DA (1 vs. 3).** Augmentation gives the **largest improvement**, +7.30 points (87.64% vs. 80.34%). The DA model reaches its best accuracy in the very last epoch, with a training loss still at 0.281: for all 30 epochs it keeps learning features that generalize instead of memorizing the training set. The baseline, by contrast, drives its training loss down to 0.048, essentially memorizing the training images, while its validation accuracy falls. The cost of augmentation is slower early fitting (56.53% after epoch 1, the lowest of the four), but by epoch 5 it has overtaken the baseline (80.30% vs. 78.67%) and it stays ahead from then on.
3. **BN and DA did not add up (3 vs. 4).** Although the two act on different things, their combination (82.47%) ends level with BN alone (82.51%) and well below DA alone (87.64%). The curves show that BN + DA reached 86.36% in epoch 26 and then dropped 3.89 points, the largest late drop of the four, so this combination fluctuates strongly late in training. Possible reasons: (a) chance, since each configuration was run with a single seed and a 5-point gap cannot be separated from noise without repeated runs; (b) BN's batch statistics vary more on augmented batches, and dropout (0.5) adds further randomness on top; (c) BN uses batch statistics in training but running averages at evaluation, and together with dropout this can widen the gap between training and evaluation behavior.
4. **Conclusion.** In this setting, data augmentation is the most effective way to improve generalization and BN comes second. The expected benefit of combining them did not appear, and its late drop should be checked with several seeds.

4. Test-set accuracy

As specified by the assignment, the 10,000-image validation set serves as the test set, and accuracy is measured with each model's final (epoch-30) weights.

Configuration	Best accuracy (epoch)	Final test accuracy
(1) Baseline	82.21% (11)	80.34%
(2) BN only	82.91% (26)	82.51%
(3) DA only	87.64% (30)	**87.64%**
(4) BN + DA	86.36% (26)	82.47%

Table 5. Accuracy on the held-out set. The best configuration is (3) DA only, at 87.64%.

5. Inference

The best model, configuration (3), was loaded from its saved weights and applied to five images drawn at random from the held-out set, using the same non-augmented transform as in evaluation. For each image it reports the predicted class and the softmax probabilities.



Figure 4. Predictions of the best model on five random held-out images.

Image	True	Predicted	P(cat)	P(dog)	Correct
1	dog	dog	0.014	0.986	✓
2	dog	dog	0.169	0.831	✓
3	dog	dog	0.005	0.995	✓
4	cat	cat	0.802	0.198	✓
5	dog	dog	0.015	0.985	✓

Table 6. Class probabilities for the five images.

Analysis. All five images are classified correctly. Four predictions have a confidence above 83% (image 3 is a dog with probability 0.995), so the model is very sure of them. Image 4, a cat, has lower confidence (0.802): the cat's posture or the background makes the model less certain, although it still answers correctly. Overall the confidence levels are consistent with the 87.64% test accuracy.

6. Summary

1. **Data.** `ImageFolder` builds the labeled datasets from the folder structure; augmentation creates label-preserving variants by random cropping and horizontal flipping, and the held-out images are never augmented.
2. **Model.** Three convolution–BN–ReLU blocks, two max-pooling layers and a 2304 → 128 → 2 head; a switch turns BN on or off, and the hand-computed parameter count (319,874) matches the model summary.

3. **Training comparison** (30 epochs each, same seed): augmentation gives the largest gain (+7.30 points) and removes the late-epoch decline; BN adds 2.17 points, mainly through steadier late training; BN + DA gives no combined benefit and has the largest late drop (3.89 points), possibly because of single-seed noise and the interaction of BN statistics with dropout.
4. **Inference.** The best model classifies all five sampled held-out images correctly, with confidence consistent with its test accuracy.